

Question 11

```
vedsk0831@DESKTOP-83FU7HB:~$ nano q11.c
vedsk0831@DESKTOP-83FU7HB:~$ gcc -Wall -Wextra q11.c -o q11
vedsk0831@DESKTOP-83FU7HB:~$ ./q11
ossp11$ ls
Command stored: ls
ossp11$ pwd
Command stored: pwd
ossp11$ history
1  ls
2  pwd
3  history
ossp11$ ls | wc
    51      51    466
ossp11$ |
```

CODE:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
#include <sys/wait.h>
```

```
#define MAX_HISTORY 10
```

```
void trim(char *str)
```

```
{
```

```
    int start = 0;
```

```
    int end = strlen(str) - 1;
```

```
    while (str[start] == ' ' || str[start] == '\t')
```

```
        start++;
```

```
    while (end >= start &&
```

```
        (str[end] == ' ' || str[end] == '\t'))
```

```

        end--;

        memmove(str, str + start, end - start + 1);
        str[end - start + 1] = '\0';
    }

int main()
{
    char *history[MAX_HISTORY];
    int count = 0;
    char command[256];

    while (1)
    {
        printf("ossp11$ ");
        fflush(stdout);

        if (fgets(command, sizeof(command), stdin) == NULL)
            break;

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
            break;

        /* Store command */
        if (count < MAX_HISTORY)
        {

```

```

    history[count] = strdup(command);
    count++;
}

/* History */
if (strcmp(command, "history") == 0)
{
    for (int i = 0; i < count; i++)
        printf("%d %s\n", i + 1, history[i]);

    continue;
}

/* Find pipe */
char *pipe_symbol = strchr(command, '|');

if (pipe_symbol != NULL)
{
    *pipe_symbol = '\0';

    char *cmd1 = command;
    char *cmd2 = pipe_symbol + 1;

    /* Remove spaces */
    trim(cmd1);
    trim(cmd2);

    int pipefd[2];

```

```
if (pipe(pipefd) == -1)
{
    perror("pipe");
    continue;
}

/* First child */
pid_t p1 = fork();

if (p1 == 0)
{
    close(pipefd[0]);

    dup2(pipefd[1], STDOUT_FILENO);

    close(pipefd[1]);

    execlp(cmd1, cmd1, NULL);

    perror("exec");
    exit(1);
}

/* Second child */
pid_t p2 = fork();

if (p2 == 0)
```

```

    {
        close(pipefd[1]);

        dup2(pipefd[0], STDIN_FILENO);

        close(pipefd[0]);

        execlp(cmd2, cmd2, NULL);

        perror("exec");
        exit(1);
    }

    close(pipefd[0]);
    close(pipefd[1]);

    waitpid(p1, NULL, 0);
    waitpid(p2, NULL, 0);
}
else
{
    printf("Command stored: %s\n", command);
}
}

for (int i = 0; i < count; i++)
    free(history[i]);

```

```
return 0;
```

```
}
```